

Documentation Architecturale ChadiBot : De la V3 à l'Échelle Industrielle (V4)

Cette documentation définit l'architecture de ChadiBot V4, un assistant IA documentaire conçu pour la transition d'un périmètre expérimental vers une infrastructure de classe industrielle. Le système traite désormais un volume critique de **1 934 705 fichiers** et **217 485 dossiers**, totalisant **2,25 To** de données. Ce document constitue le manuel technique définitif pour la reconstruction, le déploiement et la maintenance de cette infrastructure à haute disponibilité.

1. Infrastructure et Authentification Silencieuse

L'architecture impose une intégration transparente au domaine Active Directory via un mécanisme de Single Sign-On (SSO) totalement silencieux, éliminant toute friction utilisateur.

- **Reverse Proxy et Pipeline IIS** : Le flux est intercepté par un serveur IIS configuré avec le module **ARR (Application Request Routing)**. Le **DefaultAppPool** opère impérativement sous un compte **gMSA** (Group Managed Service Account), garantissant une rotation automatique des secrets et une identité réseau stable.
- **Décryptage NTLM/Kerberos** : L'identité de l'utilisateur est extraite du jeton Windows Authentication. Un script Python décode le jeton pour normaliser l'identifiant (ex: FAF-FR\abouhnac).
- **Normalisation de l'Identité** : Pour prévenir les échecs de comparaison liés à la casse, l'architecture impose la conversion systématique de l'identifiant utilisateur en **MAJUSCULES** dès l'entrée du système.
- **Schéma de Flux** : Utilisateur -> IIS (Interception Kerberos) -> Flask (Extraction/Décodage identité) -> Identification normalisée (UPPERCASE).

2. Le Scanner Massif V4 et Persistance des Données

Le passage à l'échelle a nécessité l'abandon du scripting traditionnel au profit d'appels aux APIs Windows bas niveau pour garantir la vélocité du scan.

- **Analyse de Performance** : L'approche V3 via PowerShell (Get-Acl) affichait une latence prohibitive de 0,8s par dossier. Le scanner V4 utilise os.scandir et pywin32 (APIs FindFirstFileW / FindNextFileW), atteignant une cadence de **1350 fichiers/seconde**. Le scan complet des 1,93 million de fichiers s'exécute désormais en **33 minutes**.
- **Structure de la Base de Données (chadibot_v4.db)** :

Table	Rôle	Colonnes
Clés	Détail Technique	id, chemin_dossier, acl_sids
dossiers	Cache NTFS	id, chemin_dossier, acl_sids
fichiers	Catalogue	id, dossier_id, nom_fichier, extension, est_indexable_faiss
fichiers_fts	Recherche Lexicale	nom_fichier, chemin_complet

Table virtuelle **FTS5** avec triggers de synchronisation.
- **Gestion des Chemins Longs (MAX_PATH)** : Pour outrepasser la limite historique des 260 caractères (Error 123/WinError 3), le système utilise systématiquement le préfixe `\\?\UNC\` pour les accès réseau, permettant de traiter des arborescences de 32 767 caractères.

- **Concurrence d'Accès** : L'usage du mode PRAGMA journal_mode = WAL est obligatoire pour permettre des lectures fluides via l'interface Streamlit pendant que le Webhook effectue des écritures asynchrones.

3. Le Bouclier de Sécurité NTFS Hybride

La vérification des droits est déportée du disque vers une structure de données résidant en RAM, transformant un processus I/O lourd en un arbitrage mathématique de complexité $O(1)$.

- **Résolution Dynamique du DC** : Pour éviter l'erreur **RPC 1722** (serveur indisponible), le système n'interroge pas le nom du domaine mais utilise win32net.NetGetAnyDCName pour localiser physiquement le contrôleur de domaine actif.
- **Extraction des Groupes** : Les SIDs et noms de groupes sont extraits via NetUserGetGroups. L'architecture mandate une normalisation stricte en **MAJUSCULES** pour l'intersection.
- **Mécanisme de Cache RAM** : Un dictionnaire CACHE_ACL_DOSSIERS stocke les ACLs en mémoire vive. Cela évite la saturation de la base SQLite lors de la validation des résultats de recherche.
- **Arbitrage de Sécurité** : L'accès est validé par l'intersection entre l'ensemble (set) des groupes de l'utilisateur et l'ensemble des ACLs pré-calculées en base.
- **Le Sanctuaire** : Les erreurs "Accès Refusé" (ex: dossiers RH/Direction) rencontrées lors du scan sont traitées comme une fonctionnalité de sécurité native : aucun index n'est créé pour ces zones, les rendant invisibles pour l'IA.

4. Moteur de Recherche Multi-Agents et Intelligence Artificielle

Le moteur de recherche combine trois couches technologiques pour une précision chirurgicale.

- **Couche d'Intention (Ollama)** : Le modèle Llama 3.2 extrait les mots-clés. Il possède des règles métier pour déduire les extensions (ex: "installateur" ou "setup" déclenchent une recherche sur .exe et .msi).
- **Couche Lexicale (FTS5 BM25)** : Recherche ultra-rapide sur les métadonnées.
- **Couche Sémantique (FAISS V4)** : Migration vers IndexIVFFlat avec partitionnement en cellules de Voronoï.
- **Entraînement** : L'index nécessite une phase de *Training* (au moins 39 points par cluster). Le système est entraîné sur un échantillon de 50 000 vecteurs pour garantir l'équilibre des clusters sans saturer la RAM.
- **Routeur Hybride (Extensions VIP)** : Seuls les fichiers porteurs de sens sémantique sont indexés dans FAISS.
- **Liste VIP** : .pdf, .docx, .doc, .txt, .pptx, .ppt, .md, .ps1, .py, .bat, .cmd, .msi.
- Les fichiers binaires (CAO, Images, ISO) sont uniquement catalogués en lexical (SQLite).

5. L'Arbitre RRF (Reciprocal Rank Fusion)

L'algorithme RRF fusionne les résultats des deux moteurs pour produire un classement unique.

- **Formule de Scoring** : $\text{Score} = 1.0 / (k + \text{rank} + 1)$ (avec $k=60$).
- **Pondérations Spécifiques** :

- **Bonus "God Mode" (x500)** : Appliqué aux correspondances exactes de noms de fichiers pour garantir que les fichiers nommés explicitement par l'utilisateur dominent les résultats sémantiques flous.
- **Pénalité de Mismatch (x0.01)** : Les résultats ne correspondant pas aux extensions déduites par la couche d'intention subissent une pénalité drastique.
- **Validation Finale** : L'arbitre appelle le Bouclier NTFS en dernière instance pour filtrer les documents non autorisés avant l'affichage.

6. Ingestion Asynchrone et Flux Delta

Le système assure une mise à jour en temps réel sans rescanner l'intégralité du stockage.

- **Watcher PowerShell V4** : Surveille les événements de l'OS. Le buffer est configuré à **64 Ko** pour éviter la perte d'événements lors de transferts massifs de fichiers.
- **Webhook Flask (Auto-Réparation)** : Le script `api_webhook.py` traite les événements Created, Changed, Deleted, Renamed.
- **Fonction Auto-Réparation** : Si un nouveau dossier est créé, le Webhook interroge dynamiquement le DC pour extraire ses ACLs et mettre à jour la table dossiers en temps réel.
- **Mise à jour FAISS Chirurgicale** : Utilisation des fonctions `remove_ids` et `add_with_ids` pour modifier l'index vectoriel en RAM, évitant ainsi une reconstruction totale de l'index au cours de la journée.

7. Guide de Maintenance et Runbook "Usine"

Séquence d'Allumage Opérationnelle

1. **Ollama** : Serveur LLM pour l'extraction d'intention.
2. **Webhook Python** : `api_webhook.py` (Sas de réception des deltas).
3. **Watcher PowerShell** : Surveillance active du stockage.
4. **Streamlit** : `app.py` (Interface utilisateur).

Maintenance du Cerveau

Une reconstruction complète est préconisée chaque nuit :

1. Lancement de `scanner_v4.py` pour synchroniser la structure NTFS et SQLite.
2. Lancement de `init_faiss_v4.py` pour le réentraînement des clusters et l'indexation vectorielle.

Dépannage

- **Code Erreur 1722** : Serveur RPC indisponible. Cause : Tentative d'interrogation du nom de domaine au lieu du nom du DC physique. Solution : Vérifier la résolution via `NetGetAnyDCName`.
- **Code Erreur 5** : Accès refusé. Cause : Le compte de service scanne une zone protégée (RH/Direction). Comportement : Normal, documenter l'exclusion dans les logs.
- **Performance dégradée** : Vérifier que la base SQLite est bien en mode WAL et que le buffer du Watcher est à 64 Ko.